

Key-Value Store v3

Replica primary-secondary, commit e letture stantie

Architetture dei Sistemi Distribuiti

Lezione 12

Il percorso fin qui:

- v0: contratto del nodo singolo;
- v1: concorrenza e sezioni critiche;
- v2: persistenza locale e recovery;
- v3: due nodi e replica.

La domanda nuova e': che cosa significa OK quando esistono piu' copie dello stato?

Introduciamo due ruoli distinti:

- primary: accetta le scritture dei client;
- secondary: riceve update dal primary e puo' servire letture.

La sintassi dell'API cambia poco. Cambia la semantica del commit.

Sequenza astratta:

- `ApplyPrimary` -> `ReplyOK` -> `ReplicateLater`

Conseguenze:

- scritture rapide;
- possibili letture stantie sul secondario;
- il primary puo' rispondere OK anche se il secondario e' in ritardo.

Path async nel codice

```
with self._lock:
    self._data[key] = value
self._replicate_async({
    "op": "SET",
    "key": key,
    "value": value,
})
return "OK", False
```

Interleaving possibile:

- ① il client invia SET course ads al primary;
- ② il primary risponde OK;
- ③ la replica non e' ancora arrivata;
- ④ un altro client legge GET course dal secondario;
- ⑤ il secondario risponde NOT_FOUND.

Sequenza astratta:

- `SendReplica -> WaitAck -> ApplyPrimary -> ReplyOK`

Conseguenze:

- OK piu' forte;
- maggiore latenza sulle scritture;
- se il secondario e' giu', il primary non puo' completare la scrittura.

```
with self._lock:
    replicated, message = self._replicate_and_ack({
        "op": "SET",
        "key": key,
        "value": value,
    })
    if not replicated:
        return message, False
    self._data[key] = value
return "OK", False
```


Commit locale contro commit replicato

Due nozioni da separare:

- commit locale: update presente sul primary;
- commit replicato: update presente anche sul secondario.

La variante async promette la prima, la variante sync si avvicina alla seconda.

Due letture con la stessa sintassi:

- GET course sul primary;
- GET course sul secondario;

possono osservare stati diversi.

Quindi l'API da sola non basta: conta anche da quale replica si legge.

Esempi di safety in questa tappa:

- il primary sync non promette una scrittura non ackata dal secondario;
- il secondario applica solo update provenienti dal primary;
- in assenza di nuovi update, i due nodi convergono.

Costi introdotti:

- la replica sync aumenta la latenza;
- se il secondario non risponde, il primary sync si ferma sulle scritture;
- la replica async migliora disponibilita' apparente ma indebolisce il contratto.

Esperimento 1

Con secondario lento e primary async:

- 1 SET course ads sul primary;
- 2 GET course subito sul secondario;
- 3 rilettura dopo il delay.

Obiettivo: rendere visibile il lag di replica.

Con secondario fermo:

- 1 SET course ads su primary async;
- 2 stessa operazione su primary sync.

Obiettivo: confrontare disponibilita' della scrittura e forza del commit.

Problema ancora aperto:

- cosa succede se due nodi credono entrambi di essere primary?

Questa lezione non lo risolve.

Serve per motivare la tappa successiva: failure detection e leader election.

La stessa SET cambia di nuovo significato.

Ora il contratto dipende anche da:

- quando la replica viene inviata;
- quando viene ackata;
- da quale copia arrivano le letture.

Lavoro richiesto:

- ① confrontare primary async e primary sync;
- ② rendere osservabile una stale read;
- ③ discutere il significato di commit;
- ④ collegare il problema al rischio di split brain.